

Part1: 文档中需要回答的问题:

我之前没有任何多人协同开发的经历。没有这样的经历的核心原因我觉得恰恰是分工协作不方便，因为代码这种东西你改改我改改，最后是真的不方便人工合并，太容易报错了。之前 AI 课有一个小 project 本来试图多人协同开发的，然后因为刚才说的这些东西发现还不如自己用 AI 做一整个效率高。

我觉得 Git 设计暂存-提交两步主要是解决改了半天代码发现过程中某一版功能实现的最好但是不方便回退的痛点。暂存的话就可以直接把后面那些无用功删掉，回退很方便。另外也防止了测试过程中的垃圾代码误提交，还可以方便在最终提交的时候挑选出哪些改动需要提交，哪些改动再等下一版。

git branch 和 git branch -a 的区别是:

git branch 只列出本地分支，不显示远程仓库上的分支。带标记代表当前所在分支。

git branch -a: 列出所有分支，包括本地分支和远程追踪分支以及远程仓库上的分支。名称一般以 remotes 或 origin 开头。

Part2: 网页阅读及学习 Git 原因

介绍规范的 Git 提交信息 (commit message) 格式，推荐使用结构化的提交信息，区分 feat 新功能、fix 修复 bug、docs 文档、refactor 重构等类型。规范的 commit 可以自动生成 ChangeLog，方便团队看懂每一次提交目的，快速定位历史改动。(commit message 规范)

语义化版本——主版本. 次版本. 修订号 (如 2.1.3)。主版本号不兼容 API 变更；次版本向后兼容新增功能；修订号做向后兼容 bug 修复。版本号让使用者清晰判断升级风险，大型项目、库的发布都遵循这套约定。(语义化版本)

我认为我们需要学习 Git 的原因如下:

Git 是工业界通用版本控制系统，课程、科研、实习、开发岗位几乎都会用到，多人协作开发几乎必备。它可以记录代码每一次变更，随时回滚、对比差异，防止代码丢失，并且追踪是谁在何时修改。分支机制支持并行开发，不同功能独立开发互不干扰，解决多人同时修改同一文件的冲突问题。搭配规范 commit message、语义化版本，可以让项目历史清晰可读，便于维护、发布迭代。

Part3 实验步骤:

```
PS C:\Users\zhu\Desktop\test-git\Gitlab> git commit --amen
[feature f92df1f] refactor:Turning "Hello world" to "something weird"
Date: Wed Sep 23 21:58:33 2026 +0800
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\zhu\Desktop\test-git\Gitlab> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\zhu\Desktop\test-git\Gitlab>
```

1. 初始化仓库，编写 main.c 并完成初始提交。

2. 基于当前提交创建并切换至 feature 分支，修改 main.c 中 printf 语句，提交该修改。

3. 切换回 main 分支，在 main.c 的同一代码行，修改为与 feature 分支不同的字符串，并提交，使两条分支产生分叉。

4. 在`main`分支执行`git merge feature`，触发文件内容合并冲突。

5. 打开存在冲突的 main.c，文件出现<<<<<< HEAD、=====、>>>>>> feature 冲突标记；使用冲突处理选项（接受当前更改 / 接受传入更改 / 同时接受两项更改）解决冲突。

6. 保存文件，将 main.c 加入暂存区，执行`git commit`完成合并提交。

合并发生冲突的截图：

```
M      main.c
Switched to branch 'feature'
PS C:\Users\zhu\Desktop\test-git\Gitlab> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\zhu\Desktop\test-git\Gitlab> git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

强行在源代码管理中上传合并之后选择“接受传入更改”之后的样子（即冲突解决）：

```
C main.c
C main.c (main)
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Enjoy ICS!\n");
7 }

源代码管理
存储库
Gitlab
更改
同步更改 5 个

Merge br... @main
refactor:Turning "T..."
refactor:Turning "T..."
refactor:Turning "T..."
something weird...
Init... @origin/main

问题 输出 调试控制台 终端 端口
M      main.c
Switched to branch 'feature'
PS C:\Users\zhu\Desktop\test-git\Gitlab> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\zhu\Desktop\test-git\Gitlab> git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\zhu\Desktop\test-git\Gitlab>
```